

Regioes Criticas Nomeadas e Locks Explicitos

Eugenio Vitor Lopes dos Santos

DCA3703, Programacao Paralela, UFRN

1 Enunciado

Escrever um programa que cria tarefas para realizar N insercoes em duas listas encadeadas. Cada tarefa escolhe aleatoriamente em qual lista inserir. Garantir integridade evitando condicao de corrida e, sempre que possivel, usar regioes criticas nomeadas para que a insercao em uma lista nao bloqueie a outra. Generalizar para numero de listas definido pelo usuario. Explicar por que regioes criticas nomeadas nao bastam e por que locks explicitos se tornam necessarios.

2 Fundamentacao

Inserir no inicio custa dois stores: `novo->prox = head; head = novo`. Duas threads lendo o mesmo head perdem uma insercao (*lost update*).

`critical` sem nome trava tudo num lock global unico, inclusive listas independentes. `critical(name)` separa um lock por nome: mesmo nome espera junto, nome diferente anda junto.

O nome sai resolvido em compilacao e nao aceita variavel como `critical(locks[i])`. Para N definido em execucao foi alocado `omp_lock_t locks[N]`: `omp_init_lock` antes do paralelo, `omp_set_lock/omp_unset_lock` por insercao, `omp_destroy_lock` depois. Foram criadas tasks com `parallel+single+task` e foi sorteado com `rand_r(&seed)` privada por tarefa; `rand()` divide estado global e corre com concorrencia.

3 Implementacao

Foram escritos dois arquivos no padrao v1/v2 da tarefa08 e compilados com GCC 13.3, `-std=c11 -O2 -Wall -fopenmp -lm`. Foi medido com `agora()`, sobre `clock_gettime(CLOCK_MONOTONIC)`: resolucao de nanossegundos, imune a ajustes do relógio.

Cada cronometro cobre so a regio paralela de insercoes (`parallel+single+tasks`). Alocacao, `omp_init_lock` e `omp_destroy_lock` ficam de fora. Foi fixado o escopo com `default(none)`. v1 guarda duas cabecas com `critical(lista0)/critical(lista1)`; v2 guarda `heads[nlistas]` com `locks[nlistas]`. Foram contados os nos no fim e conferido `total==N`.

```
1 gcc -std=c11 -O2 -Wall -fopenmp v1_critical_nomeado.c -o v1_critical -lm
2 gcc -std=c11 -O2 -Wall -fopenmp v2_locks.c -o v2_locks -lm
3 ./v1_critical 50000
4 ./v2_locks 50000 8
5 OMP_NUM_THREADS=4 ./v2_locks 100000 8
```

Listing 1: Compilacao e execucao

4 Resultados

Foram executados testes num i7-14700HX sob WSL2 (14 nucleos fisicos, 28 threads logicas). Foi varrido 1 a 28 threads com N = 50000 e 8 listas na v2. Cada ponto vale a mediana de 3 repeticoes. CSV completo em anexo.

```
1 [v1 critical nomeado] N=50000 c0=24998 c1=25002 total=50000 OK tempo
   =0.3221 s
2 [v2 locks] N=50000 nlistas=8 total=50000 OK tempo=0.3807 s
3 [v2 locks] N=50000 nlistas=2 total=50000 OK tempo=0.3531 s
```

Listing 2: Saidas reais de integridade

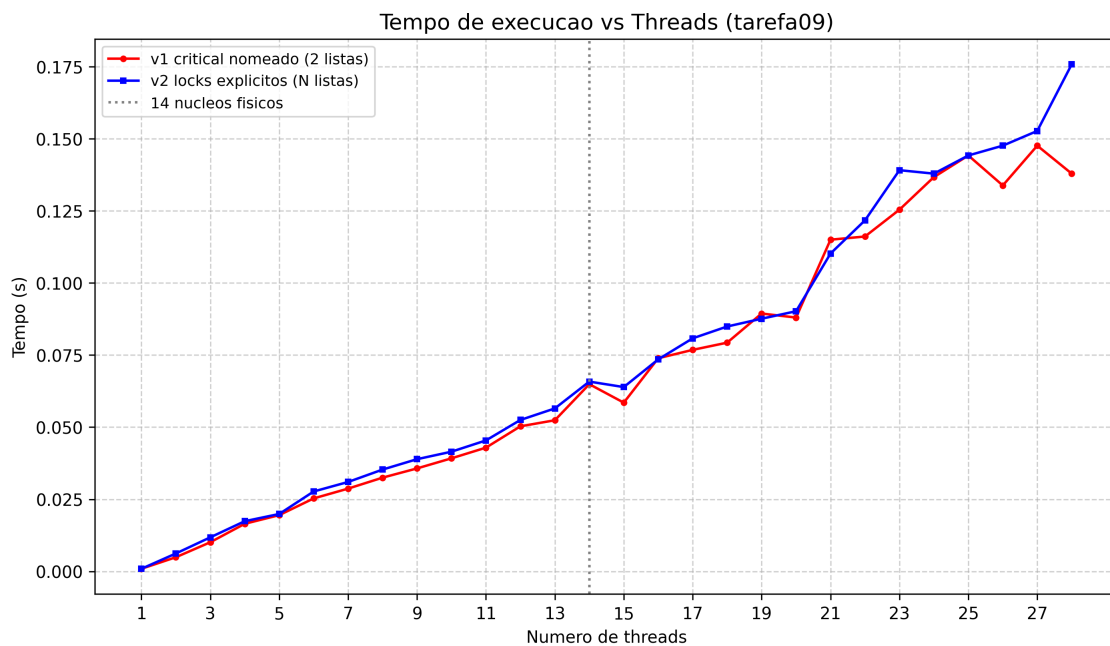


Figure 1: Tempo contra threads em escala linear.

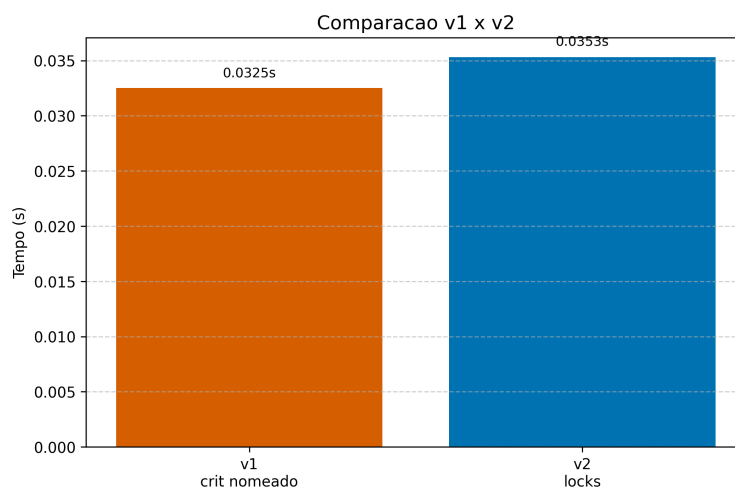


Figure 2: Comparacao v1 x v2.

5 Discussao

5.1 Sem bloqueio cruzado na v1

`critical` sem nome trava tudo num lock global: a lista 0 bloqueia a 1 sem dividir dado. Foram separados `critical(lista0)` e `critical(lista1)` em dois locks independentes. Mesma lista espera junto; listas diferentes andam juntas. Sobra so a espera inevitavel: duas tarefas na mesma lista.

5.2 Por que o nomeado nao generaliza

O nome do `critical` e literal de compilacao e nao aceita variavel:

```
1 // invalido: nao compila / nao indexa por i
2 #pragma omp critical(locks[i])
3     inserir(&heads[i], v);
```

Listing 3: Nao indexavel: nome precisa ser literal

1000 listas pediriam 1000 nomes escritos a mao, e `N` so chega em execucao via `argv[2]`. Um nome unico para tudo volta a serializar o que e independente. Foi alocado `malloc + omp_init_lock` antes do paralelo, foi travado `omp_set_lock/omp_unset_lock` por qual a cada insercao e liberado com `omp_destroy_lock` depois. Indices distintos andam juntos; mesmo indice espera. Vale a regra curta: pouco e fixo usa `critical(name)`; `N` variavel usa vetor de locks (`omp_lock_t`).

5.3 Sobre o tempo: por que sobe com threads

Cada insercao paga uma entrada em `critical/lock` mais a criacao de `N` tasks. Com `N = 20000–50000` esse custo engole o trabalho util (um `malloc + dois stores`). Mais threads somam contencao e gerenciamento sem abrir paralelismo novo: as curvas sobem. Nome e locks removem o bloqueio cruzado. A fila na mesma lista fica — custo do dado compartilhado.

6 Conclusao

Foram geradas `N` tasks com `parallel+single`: `total==N` OK nas duas versoes. Foi sorteado com `rand_r(&seed)` privada: 24998/25002, $\approx N/2$ por lista, sem a corrida do `rand()` global. Dois nomes isolam as listas na v1. `argv[2]` fixa `N` listas na v2, um `omp_lock_t` por lista. Nome nao escala porque sai resolvido em compilacao; locks indexaveis cobrem o `N` de execucao. Cada insercao paga uma entrada em `critical/lock`; privatizar nao cabe porque a lista e compartilhada.

A Código-fonte

```
1 #define _POSIX_C_SOURCE 199309L
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <omp.h>
5 #include <time.h>
6
7 typedef struct No {
8     int valor;
9     struct No *prox;
10 } No;
11
12 static double agora(void) {
13     struct timespec t;
14     clock_gettime(CLOCK_MONOTONIC, &t);
15     return t.tv_sec + t.tv_nsec / 1e9;
16 }
17
18 static void inserir(No **head, int valor) {
19     No *novo = malloc(sizeof(No));
20     novo->valor = valor;
21     novo->prox = *head;
22     *head = novo;
23 }
24
25 static long contar(No *head) {
26     long n = 0;
27     for (; head; head = head->prox) n++;
28     return n;
29 }
30
31 static void liberar(No *head) {
32     while (head) { No *t = head; head = head->prox; free(t); }
33 }
34
35 int main(int argc, char *argv[]) {
36     long total_elementos = argc > 1 ? atol(argv[1]) : 50000L;
37     No *head0 = NULL;
38     No *head1 = NULL;
39     double tempo = agora();
40     #pragma omp parallel shared(head0, head1, total_elementos) default(
41         none)
42     {
43         #pragma omp single
44         {
45             for (long i = 0; i < total_elementos; i++) {
46                 #pragma omp task firstprivate(i) shared(head0, head1)
47                 default(none)
```

```

46         {
47             unsigned int seed = 12345u + (unsigned int)(i *
48                 2654435761u);
49             int qual = (int)(rand_r(&seed) % 2);
50             int v = (int)(rand_r(&seed) % 1000000);
51             if (qual == 0) {
52                 #pragma omp critical(lista0)
53                 inserir(&head0, v);
54             } else {
55                 #pragma omp critical(lista1)
56                 inserir(&head1, v);
57             }
58         }
59     }
60 }
61 tempo = agora() - tempo;
62 long c0 = contar(head0);
63 long c1 = contar(head1);
64 printf("[v1 critical nomeado] N=%ld c0=%ld c1=%ld total=%ld %s tempo
65     =%.4f s\n",
66         total_elementos, c0, c1, c0 + c1, (c0 + c1 == total_elementos) ? "
67         OK" : "ERRO", tempo);
68 liberar(head0);
69 liberar(head1);
70 return 0;
71 }

```

Listing 4: v1_critical_nomeado.c

```

1  #define _POSIX_C_SOURCE 199309L
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <omp.h>
5  #include <time.h>
6
7  typedef struct No {
8      int valor;
9      struct No *prox;
10 } No;
11
12 static double agora(void) {
13     struct timespec t;
14     clock_gettime(CLOCK_MONOTONIC, &t);
15     return t.tv_sec + t.tv_nsec / 1e9;
16 }
17
18 static void inserir(No **head, int valor) {
19     No *novo = malloc(sizeof(No));
20     novo->valor = valor;

```

```

21     novo->prox = *head;
22     *head = novo;
23 }
24
25 static long contar(No *head) {
26     long n = 0;
27     for (; head; head = head->prox) n++;
28     return n;
29 }
30
31 static void liberar(No *head) {
32     while (head) { No *t = head; head = head->prox; free(t); }
33 }
34
35 int main(int argc, char *argv[]) {
36     long total_elementos = argc > 1 ? atol(argv[1]) : 50000L;
37     int nlistas = argc > 2 ? atoi(argv[2]) : 8;
38     if (nlistas < 1) nlistas = 1;
39     No **heads = calloc((size_t)nlistas, sizeof(No *));
40     omp_lock_t *locks = malloc((size_t)nlistas * sizeof(omp_lock_t));
41     for (int i = 0; i < nlistas; i++) omp_init_lock(&locks[i]);
42     double tempo = agora();
43     #pragma omp parallel shared(heads, locks, total_elementos, nlistas)
44         default(none)
45     {
46         #pragma omp single
47         {
48             for (long i = 0; i < total_elementos; i++) {
49                 #pragma omp task firstprivate(i) shared(heads, locks,
50                     nlistas) default(none)
51                 {
52                     unsigned int seed = 999u + (unsigned int)(i * 40503u);
53                     int qual = (int)(rand_r(&seed) % (unsigned int)nlistas);
54                     int v = (int)(rand_r(&seed) % 1000000);
55                     omp_set_lock(&locks[qual]);
56                     inserir(&heads[qual], v);
57                     omp_unset_lock(&locks[qual]);
58                 }
59             }
60             tempo = agora() - tempo;
61             long total = 0;
62             for (int i = 0; i < nlistas; i++) total += contar(heads[i]);
63             printf("[v2 locks] N=%ld nlistas=%d total=%ld %s tempo=%.4f s\n",
64                 total_elementos, nlistas, total, (total == total_elementos) ? "OK"
65                 : "ERRO", tempo);
66             for (int i = 0; i < nlistas; i++) {

```

```
66         liberar(heads[i]);
67         omp_destroy_lock(&locks[i]);
68     }
69     free(heads);
70     free(locks);
71     return 0;
72 }
```

Listing 5: v2_locks.c